Aran Learn 18: Phones, QR Codes and Speech Input

For: everyone who will run or change the “Beat the firewall” arena (`#arena` on the phone, `#arena-live` on the big screen). Inbathamizhan owns the pages; Aasif owns the challenge rules and the checker (it is a red-team tool). **Time:** about 2 hours with practice. **Before this:** Learn 8 (web APIs) and Learn 17 (wiring a design to a live API).

The arena lets judges attack Aran from their own phones. This doc teaches the five ideas it rests on: **how a phone reaches a laptop on the same Wi-Fi**, **QR codes**, **polling a shared board**, **speech recognition in the browser**, and **how to score an attack honestly**.

1. How a phone reaches your laptop

When you run `uvicorn main:app --port 8000`, the server listens on `127.0.0.1`, the **loopback** address. Only programs on the same computer can reach it. A phone is a different computer, so it cannot.

To let the phone in, listen on every network interface:

```
cd app/server
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

Now the laptop answers on its **LAN address** (for example `192.168.1.23`), the address your Wi-Fi router gave it. The phone must be on the same Wi-Fi. Find the address with `ipconfig` (Windows) or `ip addr` (Linux); the arena finds it for you (`GET /api/arena/join`).

- **College or hall Wi-Fi often blocks phone-to-laptop traffic** (“client isolation”). The fix: turn on your phone’s hotspot, connect the laptop to it, and let judges join that hotspot. Test this before the event.
- Windows may ask whether Python can accept connections on private networks. Say yes.
- Listening on `0.0.0.0` lets anyone on that network use the console. Only do it on a network you trust, and only with generated data (Aran never holds real data in the demo).

Try it: start the server with `--host 0.0.0.0`, open `http://<your-LAN-IP>:8000/#arena` on your phone.

2. QR codes

A QR code is a picture of a short text, usually a URL. The squares in three corners let the camera find and straighten the code; the rest holds the bytes plus **error correction** (Reed–Solomon codes), so a code still scans when part of it is dirty or covered.

Aran draws the code on the server with the `qrcode` Python package and returns it as SVG, so it is sharp at any size and needs no internet:

```
        import qrcode, qrcode.image.svg
        img = qrcode.make("http://192.168.1.23:8000/#arena",
image_factory=qrcode.image.svg.SvgPathImage)
```

The page shows it with `<img src="/api/qr?text=...">`. Keep the URL short: shorter text means bigger squares, which scan from further away.

3. Polling a shared board

Every phone posts its try to `POST /api/arena/try`. The server keeps the scores in memory (the `ARENA` dictionary in `standout.py`). The big screen asks `GET /api/arena/board` every 1.5 seconds and redraws.

This is **polling**: simple, and fine for a room of people. The cost is up to 1.5 s of delay and one small request per tick. The alternatives (WebSockets, server-sent events) push updates instantly but add moving parts. For a demo, polling wins.

To show only *new* tries with an animation and a sound, the page remembers the ids it has already drawn in a `Set` and marks the others `fresh`.

4. Speech recognition in the browser

Chrome, Edge and Safari offer the **Web Speech API**:

```
        const SR = window.SpeechRecognition || window.webkitSpeechRecognition
        const rec = new SR()
        rec.lang = 'ta-IN'            // Tamil (India); also 'en-IN', 'hi-IN'
        rec.interimResults = true     // show words while the person is still
speaking
        rec.onresult = (e) => { text.value = [...e.results].map((r) =>
r[0].transcript).join(' ') }
        rec.start()
```

Two rules matter:

- **The microphone only works in a secure context**: `https://` pages, or `http://localhost`. A phone opening `http://192.168.1.23:8000` is *not* secure, so the browser refuses the microphone. The arena detects this (`window.isSecureContext`) and tells the judge to use the **microphone key on the phone keyboard** instead (Gboard and the iPhone keyboard both have one). It produces the same text.
- In Chrome, recognition is done by Google's servers. Fine for a demo with generated numbers; never for real customer speech. A bank would run its own model (for example Whisper) inside its network.

Why voice matters for Aran: **bank call centres** turn calls into transcripts, and customers read out card numbers and OTPs. Spoken numbers arrive as words ("nine eight seven…"), which is exactly the disguise to test.

5. Scoring an attack honestly

A naive score would be "caught if Aran found something". But then a message with no number at all would count as a miss for Aran, and a random number would count as a catch. The arena scores against a **challenge**:

1. The server picks one generated customer's Aadhaar, card or mobile and shows it on the phone.
2. The **checker** looks for that exact number inside the judge's message, using Aran's strictest decoder (the red-team normaliser with number words, look-alike letters and double escapes turned on). If it cannot find it, the try is **not counted** ("unverified").
3. If the number is hidden in the message, the server checks what the log platform **actually stored**: if the number can still be decoded from it, the try **slipped**; otherwise it was **caught**.

The checker is stronger than the live firewall on purpose. Number words are switched off in the live firewall (they caused false alarms in E10 to E12), so "nine eight seven six…" slips. That is a real, honest result: it shows the judges a known gap, and the board records it.

Practice

1. Run the server on `0.0.0.0` and join from your phone. Send the plain number; check it is caught.
2. Spell the number in words. It slips. Read `research/firewall/normalise.py` and find the switch that would catch it, then explain in one paragraph why it is off.
3. Use your hotspot instead of the Wi-Fi and time how long a judge needs from scan to first try.

Words

- **Loopback (127.0.0.1):** the computer talking to itself.
- **LAN address:** the address on the local network, given by the router.
- **Secure context:** a page served over https or from localhost; browsers only allow the microphone and camera there.
- **Polling:** asking the server again and again on a timer.
- **Challenge:** the number a player must hide, so a try can be scored.